

Mirco-ROS 控制机械臂消除抖动

这个问题很好，抖动现象在你的架构（Movelt2 → Bridge Node → micro-ROS UDP → PCA9685 → LD-1501MG）中其实是多因素叠加的结果。让我帮你逐层分析：

抖动的主要来源

1. Movelt2 轨迹本身的问题

Movelt2 的 Action 接口（FollowJointTrajectory）输出的是一系列离散的轨迹点（waypoints）。关键在于：

- 轨迹插值的时间分辨率如果不够密，舵机在两个 waypoint 之间会出现"跳跃式"运动
- Movelt2 默认的 time_parameterization（TOTG 或 ISP）可能生成加速度突变点，传到舵机上就是抖动
- 如果你的 planning pipeline 没有做后处理平滑（如 Ruckig 平滑），轨迹本身就不够丝滑

2. UDP 传输的非确定性

WiFi UDP 是不可靠传输，这里有几个坑：

- **丢包**：某个关节的目标位置丢了，舵机保持上一个位置不动，其他关节继续走 → 抖动/撕裂感
- **乱序**：UDP 不保证顺序，如果 micro-ROS 收到的指令顺序错乱，舵机会来回跳
- **延迟抖动 (jitter)**：WiFi 的延迟不稳定（几ms到几十ms波动），导致轨迹执行的时间同步被打破

不过你之前已经加了 3 度死区滤波，这说明你其实已经意识到了这个层面的噪声。

3. PCA9685 → 舵机层面

- PCA9685 的 PWM 分辨率是 12-bit（4096 步），对应舵机的 180° 大约是 0.044°/步，但 LD-1501MG 本身的机械精度和响应速度是有限的
- 如果指令更新频率太高（比如 > 50Hz），舵机还没到位就收到新指令，会出现震颤
- 如果更新频率太低，轨迹又不连续

4. 最可能的主因：轨迹下发频率与舵机响应的不匹配

你的架构中，Movelt2 规划的轨迹是按时间戳给出的，但经过 Bridge Node → micro-ROS → ESP32 → PCA9685 这条链路后，**时间同步基本已经丢失**。ESP32 端收到指令后是"尽快执行"，而不是"在精确的时间点执行"，这就导致了轨迹的时域失真。

排查建议

你可以按这个优先级来排查和改善：

第一步：确认是不是网络问题 — 先用串口/有线方式替代 WiFi UDP 跑同样的轨迹，如果抖动消失，说明网络是主因。

第二步：检查轨迹质量 — 在 Bridge Node 端把收到的 trajectory points 打印出来，看 position 序列是否平滑，时间间隔是否均匀。如果轨迹本身就有跳变，问题在 MoveIt2 端。

第三步：在 ESP32 端加插值 — 这是最实用的改善手段。不要让 ESP32 直接"跳"到目标位置，而是在两个收到的目标之间做线性或三次样条插值，按固定频率（比如 20ms/50Hz）平滑输出给 PCA9685。本质上就是在末端加一个软件伺服环。

第四步：调整 MoveIt2 的轨迹密度 — 在 trajectory_execution 配置中增加插值密度，或者在 Bridge Node 中对轨迹做重采样，确保 waypoint 间隔足够小（比如 $\leq 50\text{ms}$ ）。